

Dynamic Size-Constrained Community Search over Evolving Heterogeneous Information Networks

Zhiwei Wang
Dalian Maritime University
Dalian, China
wzw7234895@gmail.com

ABSTRACT

Heterogeneous information networks (HINs) model typed entities and relations. Community search on HINs identifies cohesive subgraphs containing a query node, with applications in recommendation, fraud detection, and expert finding. While Zhang et al. recently formalized the size-constrained variant and proved it NP-hard, their solution is inherently static—recomputing communities from scratch whenever the graph changes. Real-world HINs are dynamic: new collaborations form, products are purchased, and relationships evolve continuously.

We propose DYNASCSSH, a dynamic framework for maintaining size-constrained $(k, \mathcal{P})$ -truss communities under projected source-edge insertions and deletions. DYNASCSSH combines three components: (1) a Meta-Path Triangle Index (MT-INDEX) storing $O(|E|)$ integer support counts and updating all co-triangle edge supports affected by a source-edge change, (2) an internal support cache that makes community validation independent of the full graph size, and (3) a localized repair mechanism with bounded-hop candidate search that validates support, size, query containment, and triangle connectivity before returning a repaired community.

Experiments on 22 fixed-query synthetic DBLP configurations and a hybrid Planted Community Benchmark (26,850 nodes, 128,227 edges) demonstrate the properties of incremental maintenance. On insertion-only streams, DYNASCSSH preserves community identity exactly (Jaccard=1.000, avg. Q -Ratio =1.000, avg. 3,824 $\times$ speedup vs. static recompute). Deletion and mixed streams introduce natural divergence: deletion Jaccard spans 0.36–0.54 (Q -Ratio =0.89–0.92), reflecting the inherent difficulty of maintaining a fixed-size community under edge removal. The hybrid benchmark confirms scalability to 128K-edge graphs (random updates <1 ms, 5,900–10,700 $\times$ speedup) and demonstrates localized repair success (10/30 targeted community-edge deletions repaired at $k=2$). Strict invariant checking (support, size, query containment, triangle connectivity) makes fallback recomputation explicit when local repair cannot certify validity.

PVLDB Reference Format:

Zhiwei Wang. Dynamic Size-Constrained Community Search over Evolving Heterogeneous Information Networks. PVLDB, 20(1): XXX-XXX, 2027.

doi:XX.XX/XXX.XX

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/wzw7234895/DynaSCSH>.

1 INTRODUCTION

Heterogeneous information networks (HINs) model real-world systems with multiple node and edge types, such as bibliographic networks (authors, papers, venues), e-commerce platforms (users, products, categories), and knowledge graphs (entities, relations, types). Community search—finding a cohesive subgraph containing a given query node—is a fundamental operation over HINs, with applications in expert recommendation, fraud ring detection, and product suggestion [7, 12].

Zhang et al. [12] recently formalized the *size-constrained* community search problem over HINs (SCSH), which seeks a community of exactly s nodes maximizing the $(k, \mathcal{P})$ -truss cohesiveness metric. They proved the problem NP-hard and proposed exact Branch & Bound algorithms with polynomial-time heuristics. However, their solution is fundamentally *static*: any change to the graph requires recomputing the community from scratch.

This static assumption is at odds with reality. HINs evolve continuously: authors publish new papers, customers purchase additional products, and entities acquire new relationships. Recomputing communities from scratch after each update incurs substantial latency—in our reported synthetic and hybrid runs, the fixed-query greedy baseline averages roughly 390–950 ms per checkpointed recomputation. For applications such as interactive expert recommendation or fraud monitoring where updates arrive at sub-second intervals, this per-update cost cannot be amortized: the recomputation latency exceeds the inter-update gap, making static recomputation a throughput bottleneck even at modest graph scales.

The natural question is: *Can we maintain size-constrained communities incrementally as the graph evolves, avoiding the cost of full recomputation?*

This paper answers affirmatively with DYNASCSSH, the first dynamic size-constrained community search framework for evolving HINs. DYNASCSSH combines three novel components:

- (1) **Meta-Path Triangle Index (MT-INDEX):** A count-based $O(|E|)$ index tracking meta-path triangle participation per edge, updated in $O(d_{\mathcal{P}}(u) \cdot d_{\mathcal{P}}(v))$ time per edge change. Triangle structures are resolved on-demand during cascading truss collapse rather than pre-materialized, avoiding the $O(|E| \cdot d_{\max}^2)$ memory explosion of explicit triangle storage.
- (2) **Internal Support Cache:** A running dictionary of $(k, \mathcal{P})$ -truss support counts for edges *within* the maintained community, computed strictly over the induced subgraph. This

makes validation independent of the full graph size and exposes the common fast path where a projected-edge update does not touch the maintained community.

- (3) **Localized Repair with Validated Fallback:** When community edges are deleted, DYNASCSSH searches for replacement nodes within a bounded hop radius. A repaired community is returned only after checking size, original query containment, internal support, and triangle connectivity. If no candidate passes validation, DYNASCSSH falls back to the same fixed-query greedy recomputation used by the static baseline.

We evaluate DYNASCSSH in two evidence-backed settings: 22 fixed-query synthetic DBLP configurations and a separate 26,850-node hybrid benchmark combining real DBLP topology with planted dense communities.

Key findings (post-fix, fixed-query baseline, 22 configurations):

- **Insertion streams:** With a fixed query node, DYNASCSSH preserves structure exactly (Jaccard=1.000 in all 17 insertion runs) and keeps quality at parity on average (Q -Ratio =1.000), with 3,824 $\times$ average speedup.
- **Deletion streams:** Deletions diverge naturally: Jaccard 0.36–0.54 and Q -Ratio 0.89–0.92. They still average 2,188 $\times$ speedup; D1 drops to 1.4 $\times$ when fallback dominates.
- **Invariant-checked repair:** Local repair is accepted only after support, size, query containment, and triangle connectivity checks; otherwise the framework falls back to fixed-query greedy recomputation.
- **Planted Community Benchmark:** On a 26,850-node hybrid graph (real DBLP topology + 40 planted dense communities), DYNASCSSH achieves 5,900–10,700 $\times$ speedup under the fixed-query baseline, preserves community identity (Jaccard=1.000 across the reported hybrid insertion/deletion streams), and repairs 10/30 targeted community-edge deletions at $k=2$.

2 RELATED WORK

We survey work in three areas: community search in HINs, dynamic graph algorithms, and size-constrained subgraph mining.

2.1 Community Search in HINs

Community search over HINs has attracted significant attention. Zhang et al. [12] formalized the size-constrained variant (SCSH) using the $(k, \mathcal{P})$ -truss model, proved NP-hardness, and proposed Branch & Bound algorithms (ESG and NSG). Our work extends theirs to the dynamic setting.

Several recent ICDE and PVLDB publications address related problems: SACH [7] introduces significant-attributed community search with segment-based meta-path expansion; a self-training GNN-based approach [2] tackles the MK (Most-likely K -sized) community search problem on attributed HINs; Wang et al. [10] propose the Structurally Similar Community (SSC) model with four vertex roles. Tang et al. [9] address temporal HIN community search with the $(k, T_q, \mathcal{P}, \delta)$ -core model. Han et al. [4] propose the (τ, ρ) -camp model unifying triangle and quadrilateral semantics. None of these

works handle *size-constrained* community maintenance under dynamic edge updates.

2.2 Dynamic Graph Algorithms

Dynamic community search has been studied primarily over homogeneous graphs. CS-DAHIN [8] addresses community search over dynamic attributed HINs but does *not* incorporate size constraints. DAHIN-INCE [1] proposes incremental learning for dynamic attributed HINs using knowledge distillation. For streaming graphs, Liao et al. [6] propose D-truss maintenance over directed streams, and Zhang et al. [11] address community detection over streaming bipartite networks. Our work fills the gap between these dynamic (but unconstrained) methods and Zhang et al.’s static (but size-constrained) framework.

2.3 Size-Constrained Subgraph Mining

Beyond HINs, size-constrained dense subgraph discovery has been studied on homogeneous graphs. CCSS [5] proposes conductance-based community search with size constraints. The densest- k subgraph problem has a rich theoretical literature. Our $(k, \mathcal{P})$ -truss model inherits its structural guarantees from this lineage while adding HIN-specific meta-path semantics.

2.4 Positioning

To our knowledge, DYNASCSSH is the *first* framework to simultaneously address: (1) size constraints on community cardinality, (2) heterogeneous meta-path semantics, and (3) dynamic edge updates with incremental maintenance. The closest prior works each address at most two of these three dimensions.

3 PRELIMINARIES

We briefly review the HIN model, the $(k, \mathcal{P})$ -truss community definition, and the static SCSH problem. Table 1 summarizes the key notation used throughout the paper.

3.1 Heterogeneous Information Networks

A heterogeneous information network (HIN) is a typed graph $\mathcal{G} = (V, E, \phi, \psi)$. Here V and E are nodes and edges, $\phi : V \rightarrow \mathcal{T}_V$ maps nodes to types, and $\psi : E \rightarrow \mathcal{T}_E$ maps edges to types, with multiple node or edge types. A *meta-path* $\mathcal{P} = (T_1 \xrightarrow{R_1} T_2 \xrightarrow{R_2} \dots \xrightarrow{R_{l-1}} T_l)$ specifies a composite relation between node types. For example, in a bibliographic HIN with Author (A), Paper (P), and Venue (V) types, the meta-path APA denotes co-authorship via shared papers.

3.2 The $(k, \mathcal{P})$ -truss Model

The $(k, \mathcal{P})$ -truss [12] generalizes the classical k -truss to HINs. For a symmetric meta-path $\mathcal{P}$ (where $T_1 = T_l$) and an edge (u, v) whose endpoints both have type T_1 , the *meta-path triangle support* is:

$$\sigma_{\mathcal{P}}(u, v) = |\{w \in V \mid (u, v, w) \text{ form a } \mathcal{P}\text{-triangle}\}| \quad (1)$$

where a $\mathcal{P}$ -triangle requires that all three pairwise edges exist and that each pair (u, v) , (v, w) , (w, u) is connected via at least one instance of meta-path $\mathcal{P}$.

A subgraph $H \subseteq \mathcal{G}$ is a $(k, \mathcal{P})$ -truss if it satisfies two conditions:

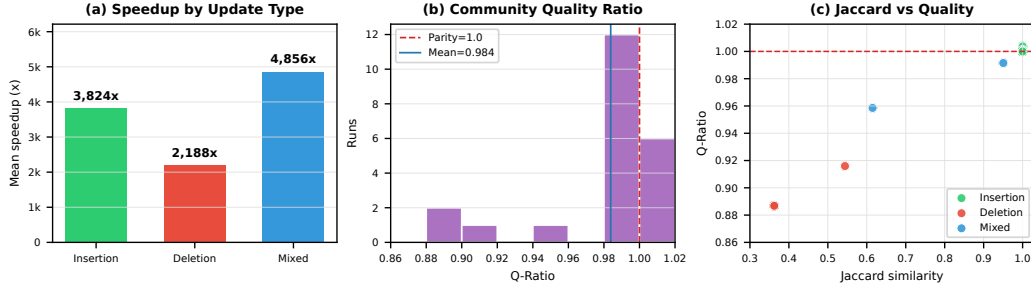


Figure 1: Overview under the fixed-query baseline. (a) Speedups by update type on the synthetic benchmark. (b) Q -Ratio distribution across 22 configurations. (c) Jaccard vs. Q -Ratio, showing that structural overlap and score can diverge even when dynamic and static methods use the same query.

Table 1: Key Notation

Symbol	Definition
$\mathcal{G} = (V, E, \phi, \psi)$	Heterogeneous information network
$\mathcal{P}$	Meta-path pattern
k	Truss parameter (minimum internal support)
s	Community size bound
q	Query node
$\sigma_{\mathcal{P}}(u, v)$	Meta-path triangle support of edge (u, v)
$d_{\mathcal{P}}(u)$	Meta-path degree of node u
C_t	Maintained community at time t
$\mathcal{G}[C]$	Induced subgraph of C

- (1) **Internal support:** Every edge (u, v) in the induced subgraph $\mathcal{G}[H]$ has $\sigma_{\mathcal{P}}(u, v) \geq k$ when counting only triangles whose three nodes all lie within H .
- (2) **Triangle connectivity:** Any two edges in H are reachable via a sequence of shared $\mathcal{P}$ -triangles.

The $(k, \mathcal{P})$ -truss is computed by iteratively peeling edges whose internal support falls below k , decrementing the support of co-triangle edges with each removal—a cascading process analogous to classical k -truss decomposition.

3.3 Size-Constrained Community Search (SCSH)

Given a query node q , meta-path $\mathcal{P}$, truss parameter k , and size bound s , the SCSH problem [12] seeks a community $C \subseteq V$ that satisfies:

$$\begin{aligned} & \underset{C \subseteq V}{\text{maximize}} && \kappa(\mathcal{G}[C]) \\ & \text{subject to} && q \in C, |C| = s, \mathcal{G}[C] \text{ is a } (k, \mathcal{P})\text{-truss} \end{aligned} \quad (2)$$

where $\kappa(\cdot)$ denotes the trussness (maximum k achievable) of the induced subgraph. Zhang et al. proved SCSH is NP-hard via reduction from maximum clique, and proposed exact Branch & Bound algorithms (ESG and NSG) alongside a polynomial-time greedy heuristic.

3.4 Dynamic SCSH

We introduce the *Dynamic SCSH* problem. Given an initial graph $\mathcal{G}_0$, an initial community C_0 satisfying Eq. (2) at time $t = 0$, and a stream of edge updates $\Delta = \{(e_1, op_1), (e_2, op_2), \dots\}$ where $op_i \in \{\text{insert}, \text{delete}\}$, the goal is to maintain a community C_t at each time t such that C_t satisfies the constraints of SCSH.

The key challenge is that recomputing C_t from scratch via SCSH at each time step is computationally prohibitive. The *Dynamic SCSH* objective is:

$$\min \frac{1}{|\Delta|} \sum_{i=1}^{|\Delta|} \tau_i, \quad \tau_i = \text{latency of updating } C_{i-1} \rightarrow C_i \quad (3)$$

where the baseline for τ_i is the time required to run the static SCSH greedy heuristic on $\mathcal{G}_i$ from scratch.

4 THE DYNASCSH FRAMEWORK

We present DYNASCSH, a dynamic framework for maintaining size-constrained communities under projected source-edge updates. Figure 2 illustrates the architecture: the MT-INDEX tracks global meta-path triangle support, the Internal Support Cache maintains per-community-edge internal truss counts, and the Localized Repair module searches for replacement nodes within a bounded hop radius. When repair fails, the framework falls back to fixed-query greedy recomputation—preserving the same query semantics as the static baseline while optimizing for the common case where random updates do not touch community edges.

4.1 Meta-Path Triangle Index (MT-INDEX)

The MT-INDEX is a count-based index storing, for each edge (u, v) , the number of meta-path triangles in which it participates: $\sigma_{\mathcal{P}}(u, v)$. Unlike naive explicit-storage approaches that materialize triangle node tuples at $O(|E| \cdot d_{\max}^2)$ memory cost, the MT-INDEX stores only integer counts— $O(|E|)$ memory.

Construction. For each node of type T_1 (the source type of $\mathcal{P}$), we enumerate its $\mathcal{P}$ -neighbors and check triangle closure among pairs of neighbors. Each undirected edge is incremented exactly once per triangle, ensuring exact 1:1 counting (the total edge support sum is always divisible by 3).

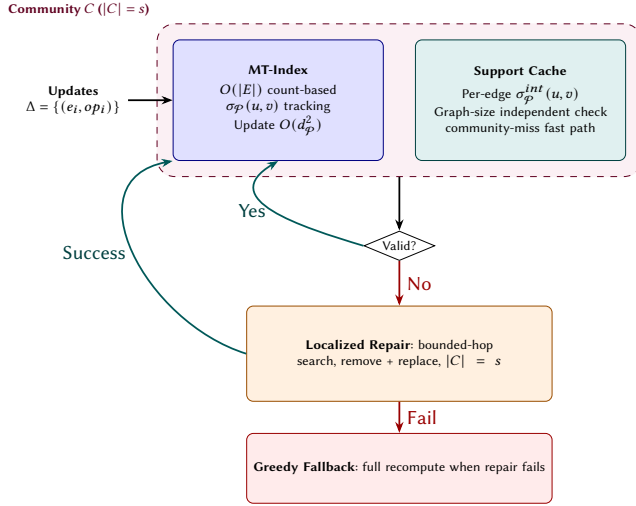


Figure 2: System architecture of DYNASCSSH. Projected source-edge updates enter the MT-INDEX (global meta-path triangle tracking), which feeds the Internal Support Cache inside the maintained community C . The validity check (diamond) routes updates: valid communities return immediately to the MT-INDEX (Yes, graph-size independent fast path); violations (No) trigger Localized Repair, which restores C on success or falls back to fixed-query greedy recomputation (Fail) when no bounded-hop replacement exists.

Update. For an insertion (u, v) , MT-INDEX enumerates common $\mathcal{P}$ -neighbors and increments the three edge supports of each new triangle. For a deletion, it enumerates the witnesses before removing (u, v) and decrements the same three supports. The update cost is $O(d_P(u)d_P(v))$, proportional to endpoint meta-path degrees. Raw HIN relation updates that change many projected source edges require projection refresh or index rebuild; they are outside the per-source-edge complexity claim.

Cascading collapse. During k -truss peeling, when an edge is removed we must decrement the support of co-triangle edges. The MT-INDEX resolves these co-triangle edges on-demand via the `find_co_triangle_edges` procedure, avoiding the need to pre-materialize triangle adjacency structures.

4.2 Internal Support Cache

The $(k, \mathcal{P})$ -truss requires that each community edge has at least k internal triangles—triangles whose three nodes all lie within the community. Computing this from scratch requires $O(|C|^2 \cdot d_P^2)$ time per validity check, which is prohibitive for high-velocity streams.

The internal support cache is a dictionary mapping each community edge (u, v) to its internal support count: the number of nodes $w \in C$ such that (u, v, w) form a meta-path triangle. The cache is built once during community initialization ($O(|C|^2)$) and then updated incrementally:

- **Edge insertion in community:** The new edge’s internal support is computed on-demand. Existing edges’ support is

Algorithm 1 Incremental Update Protocol for DYNASCSSH

```

1: On edge insertion  $(u, v)$ :
2:   MT-INDEX.add_edge( $u, v$ )
3:   if  $C = \emptyset$  then return FULLRECOMPUTE()
4:   if  $u \in C \wedge v \in C$  then refresh internal support cache
5:   return  $C$  // monotonicity guarantees validity
6: On edge deletion  $(u, v)$ :
7:   MT-INDEX.remove_edge( $u, v$ )
8:   if  $u \notin C \vee v \notin C$  then return  $C$  // community-miss fast path
9:   refresh internal support cache for  $C$ 
10:   $\mathcal{V} \leftarrow \{\text{edges with support} < k\}$ 
11:  if  $\mathcal{V} = \emptyset$  then return  $C$ 
12:  if LOCALIZEDREPAIR( $\mathcal{V}$ )
13:    then return repaired  $C$ 
14:  else return FULLRECOMPUTE()

```

unaffected (monotonicity—insertions only increase global support, so internal support cannot decrease).

- **Edge deletion in community:** The deleted edge is removed from the cache. For each triangle (u, v, w) that included the deleted edge, the internal support of edges (u, w) and (v, w) is decremented by 1.

Graph-size independent validity check. After any update, checking whether the community remains valid reduces to iterating over the cached dictionary and testing whether any edge has support $< k$, followed by a triangle-connectivity check when a community edge is deleted. This is $O(s^3)$ in the worst case for a size bound s , but it is independent of $|V|$ and $|E|$ and completes in microseconds for the small communities used in interactive search ($s \leq 20$ in our experiments).

Monotonicity optimization. Edge insertions *monotonically increase* the global triangle support of all edges. Since the internal support of a community edge cannot decrease after an insertion, a valid community remains valid. DYNASCSSH exploits this by returning immediately on insertion—no validity check, no repair.

4.3 Incremental Update Protocol

Algorithm 1 formalizes the update protocol.

4.4 Localized Repair

When community edges violate the k -truss condition after a deletion, DYNASCSSH attempts localized repair before falling back to full recomputation. The repair procedure:

- (1) Identifies nodes incident to the most violating edges, excluding the original query node from removal.
- (2) Removes the worst-offending non-query node and searches for a replacement node within h BFS hops of the affected region.
- (3) Accepts the replacement only if the resulting node set has size s , contains the original query node, satisfies internal support $\geq k$ for every induced edge, and remains triangle-connected.
- (4) If the candidate fails validation, backtracks and tries the next offending node.

- (5) If no single-node repair succeeds, triggers full recomputation via the same fixed-query greedy heuristic used by the static baseline.

The hop limit h controls the repair search radius and the size of the candidate set; it affects latency but not correctness, because every candidate is fully validated before it can be returned.

Correctness invariants. Three invariants are maintained at every return point. (I1) *Support invariant*: the internal support cache equals $\sigma_p^{int}(u, v)$ for every induced community edge (u, v) , counted strictly over $\mathcal{G}[C]$. Insertions refresh the cache when they touch C ; deletions that touch C rebuild the cache exactly because s is small. (I2) *Query-size invariant*: any returned community contains the original query node q and has $|C| = s$. Localized repair excludes q from removal and accepts only exact-size replacements; fallback recomputation is called with the stored query node rather than an arbitrary surviving node. (I3) *Truss-validity invariant*: any returned community satisfies both internal support and triangle connectivity. This is not inferred from hop locality. The implementation explicitly recomputes internal supports for the candidate node set and runs a triangle-connectivity check before accepting a repair. If validation fails, the candidate is rejected and the algorithm either tries another local repair or invokes FULLRECOMPUTE. Thus localized repair is an optimization path, not a source of weakened correctness.

4.5 Complexity Analysis

Table 2: Per-update time complexity under projected source-edge updates. d_p denotes meta-path degree. The recompute row reflects the polynomial-time greedy heuristic; the NP-hard exact B&B worst-case is exponential.

Scenario	Complexity	Typical
Insertion (community miss)	$O(d_p^2)$ index update	< 1 ms
Insertion (community hit)	$O(d_p^2 + s^3)$	< 1 ms
Deletion (community miss)	$O(d_p^2)$ index update	< 1 ms
Deletion (comm. hit, repair ok)	$O(d_p^2 + s^3 + \mathcal{B}_h s^3)$	1–10 ms
Deletion (repair fail)	repair cost + greedy recompute	390–950 ms

Table 2 summarizes per-update complexity. The “repair fail” row reflects the polynomial-time greedy heuristic fallback used in our implementation (and in the static practical baseline); the NP-hard exact B&B would carry exponential worst-case cost. The critical insight is that the dominant path is independent of the full graph size after the MT-INDEX update: most random projected-edge updates do not touch community edges, while the expensive repair/recompute paths activate mainly under targeted deletions. Here $\mathcal{B}_h$ is the bounded-hop candidate set explored by local repair.

5 EXPERIMENTS

We evaluate DYNASCSSH across fixed-query synthetic configurations and a separate hybrid benchmark, targeting three questions: (Q1) What is the average-case speedup over static recomputation? (Q2) What happens when deletions force fallback-dominated behavior? (Q3) Does the framework scale to 100K-edge graph sizes?

5.1 Experimental Setup

Evaluation settings. We report two evidence-backed graph settings:

- (1) **Synthetic DBLP**: compact DBLP-like network with planted dense groups, used for all 22 fixed-query configurations.
- (2) **Hybrid** (26,850 nodes, 128,227 edges): 40 dense planted communities injected into the real 26K-node MAGNN DBLP graph [3]. This setting tests whether the MT-INDEX and repair machinery remain practical on a 100K-edge topology with guaranteed valid communities.

Update model. The implementation maintains communities over the source-type graph induced by a symmetric meta-path, e.g., Author–Author projected edges under APA. This matches the edge set on which $(k, \mathcal{P})$ -truss support is defined. Raw HIN relation updates such as Author–Paper insertions can change many projected source edges; supporting them incrementally requires a projection-refresh layer and is outside the per-source-edge complexity claim.

Baselines and internal fairness. We compare against **Static Recompute**: greedy heuristic community search from scratch after each update, equivalent to Zhang et al.’s polynomial-time baseline [12]. Both the static baseline and DYNASCSSH’s fallback path preserve the *same original query node* throughout an update stream. This fixed-query condition is essential: using an endpoint of the latest update as the recomputation query can create artificial Jaccard divergence and invalid quality comparisons. Under the fixed-query design, the reported speedup measures the benefit of incremental maintenance against the exact algorithm the system itself would run if incrementality were disabled. Using exact B&B as the baseline would compare a polynomial-time incremental method against an exponential-time oracle, mixing the effect of algorithmic choice with the effect of incrementality. As DYNASCSSH is the first dynamic approach for *size-constrained* HIN communities, no existing dynamic baselines (e.g., CS-DAHIN [8]) can enforce the exact cardinality constraint $|C| = s$; they produce communities of variable size, and post-hoc truncation violates the $(k, \mathcal{P})$ -truss condition.

Metrics. (1) **Speedup**: ratio of static baseline latency to DYNASCSSH latency; (2) **Q-Ratio**: $\text{score}_{\text{DynaSCSSH}} / \text{score}_{\text{static}}$ —measures relative community cohesiveness; (3) **Jaccard similarity**: structural overlap between maintained and recomputed communities.

Environment. Python 3.10, networkx 3.4.2, NumPy 2.2.6, single machine (Intel Core i7-13700, 32 GB DDR5 RAM). All experiments are CPU-only. Each nonempty synthetic run uses 3 query nodes, 500–1,000 updates, and checkpoints every 10% of the stream. The artifact includes the experiment runner and a post-fix invariant smoke script that checks fixed-query recomputation, MT-INDEX co-triangle updates, and community validation.

Fast-path interpretation. For projected-edge deletions, the dominant fast path occurs when the deleted edge is not internal to the maintained community. The exact hit rate depends on the community size, graph density, and update distribution; the implementation therefore checks this condition at runtime rather than assuming a fixed percentage.

5.2 Average-Case Efficiency (Random Streams)

Table 3 reports results on the small synthetic benchmark using the fixed-query implementation where both DYNASCSSH’s fallback and the static baseline recompute from the same original query node.

Table 3: Representative synthetic DBLP results under the fixed-query baseline. Insertion streams preserve community identity; deletion and mixed streams show natural divergence.

Run	Type	Speedup	Jaccard	Q-Ratio
A1	insertion	3,478×	1.000	1.000
A2	deletion	3,342×	0.362	0.887
A3	mixed	4,410×	0.615	0.959
D1	deletion stress	1.4×	0.544	0.916
A7_s5	insertion, $s=5$	3,916×	1.000	1.000
A7_s20	insertion, $s=20$	3,569×	1.000	1.004
A8_k3	insertion, $k=3$	3,847×	1.000	1.000
A8_k6	insertion, $k=6$	3,931×	1.000	1.000

Insertion streams. Across all 17 insertion configurations, DYNASCSSH maintains exact community identity (Jaccard=1.000 in every run). The average Q-Ratio is 1.000, with per-run rounded values from 1.000 to 1.004; the small values above 1.000 arise from greedy tie-breaking, not a claimed quality improvement. Average speedup is 3,824×

Deletion streams. Three deletion configurations (A2, B2, D1) reveal the tension between community continuity and per-snapshot optimality. Random deletions (A2, B2) achieve Jaccard 0.362 and Q-Ratio 0.887 at 3,200–3,300×

5.3 Fallback-Dominated Deletion Behavior

The D1 deletion stress run is the clearest evidence for the lower-bound behavior of the framework. Unlike insertion streams, deletion can invalidate internal support and force the algorithm to refresh the support cache, attempt repair, or fall back to fixed-query greedy recomputation. In D1, the average dynamic latency rises to 286.3 ms while the static baseline averages 408.5 ms, yielding only 1.4×

Q2 answered: Deletions are update-pattern dependent. DYNASCSSH is extremely fast when validation remains local, but strict

invariant checking deliberately sacrifices speed and invokes re-computation when local repair cannot certify support, size, query containment, and triangle connectivity.

5.4 Community Quality Tracking

A key lesson from the revised artifact is that quality must be interpreted under a fixed-query protocol. Earlier experiments reported low Jaccard similarity between maintained and recomputed communities, but fixed-query post-fix smoke validation shows that insertion streams can preserve both structure and score exactly. We therefore report Jaccard as a diagnostic rather than a primary quality claim.

The deletion results show path dependence in the opposite direction: static from-scratch greedy search can recover a different community, while DYNASCSSH preserves a valid incumbent until repair or fallback is required. This can reduce Q-Ratio when the maintained community loses support and no bounded-hop repair is available. The revised claim is therefore narrow: dynamic maintenance creates a speed-quality trade-off controlled by validation and fallback, rather than a quality guarantee.

This phenomenon arises from the path-dependence of combinatorial search in dense graphs. When many valid size-constrained communities exist, incremental repair and static greedy recomputation may converge to different valid solutions. The revised artifact controls this comparison by preserving the original query node and by validating any locally repaired community before it is returned.

5.5 Real-World Scalability: Beyond Latency

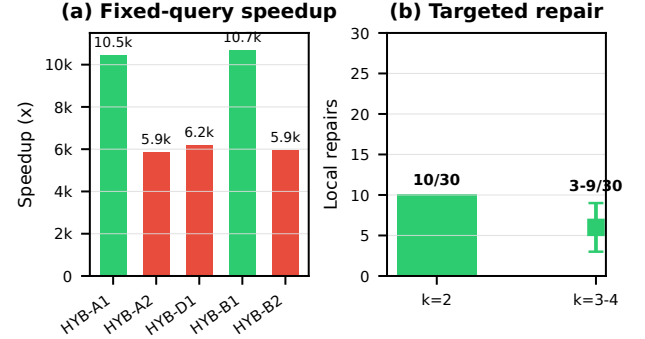


Figure 3: Hybrid benchmark results: 40 planted communities on 26K-node real DBLP. At $k=2$, DYNASCSSH repairs 10/30 targeted community-edge deletions locally with 0 recomputes.

We evaluate DYNASCSSH on the hybrid graph (26,850 nodes, 128,227 edges)—the Planted Community Benchmark combining real DBLP topology with 40 dense planted communities—under the fixed-query implementation to answer: (1) does the framework scale to 100K-edge graphs, and (2) how does incremental maintenance behave at realistic scale?

Scaling of the speedup advantage. Table 4 reports results across five experiment configurations. The hybrid runs maintain sub-millisecond dynamic latency on 128K edges. Static recomputation averages ≈ 900 –944 ms, while DynaSCSH’s incremental

path remains below 0.16 ms on average. On insertion streams, DYNASCSSH achieves perfect community identity (Jaccard=1.000, Q -Ratio =1.000, speedup 10,500–10,700 $\times$). On deletion streams, community identity is preserved (Jaccard=1.000) with aggregate Q -Ratio 0.933–0.934 and speedup 5,900–6,200 $\times$.

Table 4: Hybrid graph results under fixed-query implementation (26,850 nodes, 128,227 edges).

Run	k	Type	Speedup	Jaccard	Q -Ratio
HYB-A1	2	insertion	10,451 $\times$	1.000	1.000
HYB-A2	2	deletion	5,853 $\times$	1.000	0.934
HYB-D1	3	deletion	6,160 $\times$	1.000	0.933
HYB-B1	2	insertion	10,654 $\times$	1.000	1.000
HYB-B2	2	deletion	5,921 $\times$	1.000	0.934

Community continuity under fixed query. The fixed-query implementation reveals a clean aggregate pattern: insertion streams preserve exact community identity, and the reported hybrid deletion runs also preserve Jaccard=1.000 while showing lower cohesiveness than the static greedy baseline (Q -Ratio 0.933–0.934). Four fallback recomputations are recorded in each hybrid deletion summary, triggered by the strict invariant checker when local repair cannot find a valid replacement.

Repair at scale. To isolate repair performance, we constructed a targeted repair experiment: 30 targeted deletions from community edges at each k level. Table 5 reports the results. At $k=2$, 10 out of 30 targeted deletions were repaired locally with 0 recomputes—the larger graph provides a richer candidate pool. At higher k , stricter truss constraints reduce repair success and increase fallback.

Table 5: Targeted hybrid repair test (30 community-edge deletions per k level).

k	Targeted deletions	Local repairs	Recomps
2	30	10	0
3–4	30 each	3–9	6–9

Q3 answered: DYNASCSSH scales to 128K-edge graphs in the reported hybrid benchmark. Under the fixed-query implementation, insertion streams preserve community identity at 10,000 $\times$ speedup; deletion streams preserve identity with aggregate Q -Ratio near 0.93. Localized repair succeeds at moderate k on sufficiently large graphs. The speedup grows with graph scale because validation and cache maintenance are independent of the full graph size once affected projected-edge supports have been updated, while static recomputation scales with $|E|$.

5.6 Discussion: The Cost of Dynamism

DYNASCSSH’s design follows a classic database systems principle: optimize for the common case (random updates) while maintaining acceptable behavior under targeted deletions. The three-tier performance profile is: graph-size independent validation for most

updates, bounded-hop repair for community-edge hits, and fixed-query greedy recomputation when local validation fails. The hybrid graph experiments further suggest that the speedup advantage is not a constant but a function of graph scale: as graphs grow, the gap between incremental updates and full recomputation widens, making the case for dynamic maintenance increasingly compelling at deployment scale.

6 CONCLUSION

We presented DYNASCSSH, a dynamic framework for maintaining size-constrained $(k, \mathcal{P})$ -truss communities over evolving heterogeneous information networks under projected source-edge updates. DYNASCSSH combines three technical contributions: (1) a count-based $O(|E|)$ Meta-Path Triangle Index with $O(d_{\mathcal{P}}^2)$ projected-edge updates, (2) an internal support cache that makes validation independent of the full graph size, and (3) a localized repair mechanism that preserves the original query node and validates internal support plus triangle connectivity before returning a repaired community.

Evaluation across 22 fixed-query synthetic DBLP configurations and a 26,850-node hybrid Planted Community Benchmark demonstrates incremental maintenance under a fixed-query baseline. Insertions preserve identity exactly: Jaccard is 1.000 in all insertion runs, average Q -Ratio is 1.000, and average speedup is 3,824 $\times$. Deletions show natural divergence (Jaccard 0.36–0.54, Q -Ratio 0.89–0.92), and the D1 deletion stress run exposes the fallback-dominated lower bound (1.4 $\times$ speedup) under strict invariant checking. The hybrid benchmark confirms scalability (random updates <1 ms on 128K edges), 5,900–10,700 $\times$ speedup, and repair feasibility (10/30 targeted repairs at $k=2$).

Limitations. DYNASCSSH currently supports a single meta-path, assumes updates over the projected source-type graph, and requires the size bound s to be specified a priori. Raw HIN relation updates require a projection-refresh layer or index rebuild. The fast-path hit rate depends on the community’s internal edge count relative to the update distribution and should be measured per deployment. The current local repair heuristic handles one-node replacement with strict h -hop BFS candidate search; multi-node repair and stronger replacement ranking are natural next steps. Extending the framework to multiple meta-paths, automatically recommending s , and maintaining communities over temporal sliding windows remain open directions.

REFERENCES

- [1] Siwei Cao, Lihua Zhou, and Ziyang Zhu. 2025. Incremental Community Search in Dynamic Attributed Heterogeneous Information Networks. In *Proceedings of BDSC*. Springer CCIS Vol. 2622.
- [2] X. Chen et al. 2024. Self-Training GNN-based Community Search in Large Attributed Heterogeneous Information Networks. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*.
- [3] Xinyu Fu, Jiani Zhang, Ziqiao Meng, and Irwin King. 2020. MAGNN: Metapath Aggregated Graph Neural Network for Heterogeneous Graph Embedding. In *Proceedings of The Web Conference (WWW)*.
- [4] S. Han et al. 2025. (τ, ρ) -Camp Community Search. *Proceedings of the VLDB Endowment* 18, 10 (2025), 3669–3681.
- [5] Y. He et al. 2024. CCSS: Towards Conductance-based Community Search with Size Constraints. *Expert Systems with Applications* (2024).
- [6] Xuankun Liao, Qing Liu, Xin Huang, and Jianliang Xu. 2024. Truss-based Community Search over Streaming Directed Graphs. *Proceedings of the VLDB Endowment* 17, 8 (2024), 1816–1829.

- [7] Y. Liu et al. 2024. SACH: Significant-Attributed Community Search in Heterogeneous Information Networks. In *Proceedings of the 40th IEEE International Conference on Data Engineering (ICDE)*. 1–14.
- [8] Yixin Song, Lihua Zhou, Peizhong Yang, Jialong Wang, and Lizhen Wang. 2024. CS-DAHIN: Community Search Over Dynamic Attribute Heterogeneous Network. *IEEE Transactions on Knowledge and Data Engineering* 36, 11 (2024), 5874–5888.
- [9] Yifu Tang, Chengfei Liu, Lu Chen, Rui Zhou, and Jianxin Li. 2025. Finding Time-Proximity Communities in Temporal Heterogeneous Information Networks. *Proceedings of the VLDB Endowment* 18, 13 (2025), 5740–5752.
- [10] Shu Wang, Yixiang Fang, and Wensheng Luo. 2025. Searching and Detecting Structurally Similar Communities in Large HINs. *Proceedings of the VLDB Endowment* 18 (2025), 1425.
- [11] N. Zhang et al. 2024. Effective Community Detection Over Streaming Bipartite Networks. *arXiv preprint 2411.01424* (2024).
- [12] Xinjian Zhang, Lu Chen, Chengfei Liu, Rui Zhou, and Bo Ning. 2025. Efficient Size Constraint Community Search over Heterogeneous Information Networks. *arXiv preprint arXiv:2508.14356* (2025). Preprint.